Осигуряване на качеството (Quality Assurance - QA) е съвкупност от дейности, проектирани да оценят процеса, чрез който се създават и поддържат софтуерните продукти. Неговата основна цел е да докаже правилното поведение на системата и да установи механизми за откриване и отстраняване на проблеми.

Качество на софтуера

Качеството се дефинира като степента, до която системата или нейните компоненти отговарят на специфицираните изисквания и удовлетворяват очакванията на заинтересованите лица. То може да се разглежда от две гледни точки:

- **Съответствие с изискванията:** Реализиране на описаните функционални и нефункционални параметри.
- **Удовлетворяване на потребителските нужди:** По-широк поглед, включващ полезност и надеждност в конкретен контекст, дори ако първоначалните изисквания са били непълни.

Алтернативи за осигуряване на качеството

1. **Ориентирана към процеса (Софтуерна разработка):** Фокус върху превенцията на дефекти чрез ясни изисквания, стандарти за кодиране и ревюта.
2. **Ориентирана към продукта (Софтуерна поддръжка):** Фокус върху динамичното тестване и поддръжката на вече готовия продукт.

Тестването е процес на изпълнение на софтуера с цел откриване на дефекти. Важно е да се отбележи, че тестването доказва наличието на грешки, но не може да гарантира тяхното пълно отсъствие.

Основни тестови дейности

- **Планиране на тестовете:** Дефиниране на цели, стратегия, ресурси и критерии за край.
- **Конструиране на тестов модел:** Анализ на информацията и създаване на модел за тестване.
- **Генериране на тестови сценарии (Test cases):** Колекция от входове и очаквани резултати.
- **Управление на тестови пакети (Test suites):** Групиране на сценарии, които се изпълняват в последователност.
- **Изпълнение и наблюдение:** Стартиране на тестовете и събиране на резултати чрез "тестови оракули".
- **Анализ и проследяване:** Идентифициране на повреди и управление на дефекти (регистрация и приоритизация).

Управление на екипите

- **Вертикален модел:** Организацията е около конкретен продукт.
- **Хоризонтален модел:** Един екип извършва специфичен тип тестване за всички продукти в голяма организация.
- **Смесен модел:** Ниските нива на тестване са към проектите, а системното тестване се споделя.

Автоматизация на тестването

Целта е освобождаване на тестерите от повтарящи се задачи (например регресионно тестване). Автоматизацията има висока цена за поддръжка и не заменя човешката преценка при изследователско тестване.

Тестова дейност	**Възможност за автоматизация**
Изпълнение на тестове	Висока
Анализ на надеждността и покритието	Висока
Генериране на тестови сценарии	Висока
Създаване на формални модели	Средна
Цялостно планиране и подобряване на продукта	Ниска

Тестване с контролни списъци (Checklists)

Това е надграждане на **Ad-hoc тестването** (свободно наблюдение). Използват се неформални списъци ("TODO") за проследяване. Видовете списъци биват базови, йерархични, комбинирани и смесени. Основен проблем е трудността при описване на сложни взаимодействия.

Тестване с класове на еквивалентност

Идея: Входният домейн се разделя на класове, за които се очаква системата да се държи идентично. Избираме по 1 тест от всеки клас → пълно покритие с минимум тестове.

Стъпки:

1. Дефиниране на класове на еквивалентност (валидни и невалидни)
    - **Валидни класове:** Данни, които системата трябва да приеме.
    - **Невалидни класове:** Данни, които трябва да предизвикат грешка или отказ.
2. Избор на 1 тестови сценарий за всеки клас
3. Постигане на пълно покритие

Типове разделяне:

- Базирано на **софтуерни елементи** (полета, бутони)
- Базирано на **свойства**, релации, логически условия
- Комбинирано

Пример: Поле "Възраст" с валидни стойности 0–120:
Клас 1 (невалиден): < 0 → тест: -1
Клас 2 (валиден): 0–120 → тест: 50
Клас 3 (невалиден): > 120 → тест: 150

Разделяне на входния домейн и тестване на границите

Идея: Грешките се концентрират около граничните стойности на класовете.

Стъпки:

1. Идентифициране на входното пространство + дефиниране на домейн
2. Разделяне на поддомейни
3. Анализ на границите по всички измерения
4. Избор на тестови точки: граница - 1, на границата, граница + 1
5. Тестване и анализ

Гранични тестови точки (за домейн [a, b]):
a - 1 (точно под долната граница)
a (долна граница)
a + 1 (точно над долната)
b - 1 (точно под горната)
b (горна граница)
b + 1 (точно над горната)

Проблеми при определяне на граници:

- Грешка на затвореността (отворена граница → затворена)
- Изместване на границата
- Липсваща граница
- Излишна граница

Други техники

- **Тестване по двойки (Pairwise testing):** Използване на ортогонални масиви за намаляване броя на тестовете при много комбинации.
- **Таблици на решенията:** Използват се при сложна логика с комбинации от условия (True/False) и съответни действия.

Тестовете са организирани йерархично според нивото на абстракция:

1. **Тестване за приемане (Acceptance testing):** Проверка на готовността за реална експлоатация. Включва Алфа (в контролирана среда) и Бета тестване (от реални потребители).
2. **Системно тестване (System testing):** Тестване на цялата система като едно цяло по метода на "черната кутия" срещу функционалните изисквания.
3. **Интеграционно тестване (Integration testing):** Проверява взаимодействията между модулите. Използва се както "бяла", така и "черна кутия".
4. **Тестване на ниво програмна единица (Unit testing):** Най-ниското ниво. Проверява отделни функции или класове. Извършва се от разработчици чрез метода на "бялата кутия".



Ниво	Метод	Изпълнява се от	Описание
Unit Testing	Бяла кутия	Разработчиците	Функция, метод или клас изолирано
Integration Testing	Бяла + Черна	QA/Dev	Взаимодействие между модули
System Testing	Черна кутия	QA	Цялата система като едно цяло
Acceptance Testing	Черна кутия	Клиент/QA	Готовност за доставка; в Scrum — в края на Sprint

Бяла кутия (White-box): Тестерът познава вътрешната реализация.
Черна кутия (Black-box): Тестерът вижда само I/O; не познава вътрешността.

Допълнителни видове:

- **Beta Testing** — тестване от реални потребители в real environment
- **Regression Testing** — проверка, че нови промени не са счупили съществуващо
- **Performance Testing** — бързодействие под очаквано натоварване
- **Load Testing** — поведение под максимално натоварване

Измерването е процес на съпоставяне на стойности към характеристиките на обекта.

Характеристики на метриките

- **Надеждност:** Еднозначни резултати при еднакви условия.
- **Валидност:** Измерва точно това, което се изисква.
- **Практичност:** Лесно събиране и анализ на данните.

Класификация на метриките

- **Процесни метрики:** Ефективност на разработката (напр. брой открити дефекти на етап, средно време за изпълнение).
- **Продуктови метрики:** Качество на софтуера (напр. MTTF – средно време до отказ, MTTR – средно време за възстановяване, процент грешки, удовлетвореност на потребителите).
- **Проектни метрики:** Управление на ресурси, време и разходи.